webSPDZ + ATLAS: changes needed to make it work

Running `surplus_matching_private` with 4 browser players · Linux · 2026-10-06

1. Summary

Following the "Surplus + ATLAS" setup guide with the newest Emscripten failed in three different places. Each failure came from the build environment, not from the MPC program or the inputs. The final working setup needs only **two changes**:

> **Change A:** pin Emscripten (emsdk) to version **4.0.6**.
> **Change B:** compile everything with WebAssembly exception support. Put one line in `CONFIG.mine`: `MY_CFLAGS += -fwasm-exceptions`

No source files of webSPDZ / MP-SPDZ were modified. The `Makefile` and `CONFIG` are unchanged.

2. Working environment (verified)

Component	Version / value
webSPDZ repository commit	`3072e5a7` (branch `main`)
Emscripten (emsdk)	**4.0.6** (`1ddaae4d2d6dfbb678ecc193bc988820d1fc4633`)
Node (bundled with emsdk)	node-24.19.0-64bit (system `node`: v24.18.0)
Python	3.12.3
Build target	`WEB = 1`, wasm64 (`-sMEMORY64`), prebuilt libs in `local/wasm64/lib`
`CONFIG.mine`	`MY_CFLAGS += -fwasm-exceptions`
MPC program	`Programs/Source/surplus_matching_private.mpc`, compiled with plain `./compile.py surplus_matching_private`
Browser	Google Chrome (DevTools 152)

3. Problems found and their fixes

Problem 1: link error `recompile with -fPIC` (Emscripten 6.0.10)

Symptom (during `make -B atlas-party.x`):

```
wasm-ld: error: .../local/wasm64/lib/libgmp.a(lt4-clear.o): relocation
R_WASM_MEMORY_ADDR_LEB64
```

```
cannot be used against symbol `__gmp_free_func`; recompile with -fPIC
wasm-ld: error: .../local/wasm64/lib/libcrypto.a(asn1_lib.o): relocation
R_WASM_MEMORY_ADDR_SLEB64 ...
make: *** [Makefile:183: libwebSPDZ.so] Error 1
```

Cause: `Makefile:182-183` builds `libwebSPDZ.so` with `emcc -shared`. Up to Emscripten 5.x, `-shared` only produced a merged object file (a "fake" shared library, `FAKE_DYLIBS=true`). Since **Emscripten 6.0.0**, `FAKE_DYLIBS` is off by default, so `-shared` builds a real side module. That requires all code to be position-independent. The prebuilt `libgmp.a` and `libcrypto.a` in the repo (committed in 2024) are not compiled with `-fPIC`.

Fix: use an Emscripten version older than 6.0.0 (first 5.0.7, finally 4.0.6, see Problem 2).

Problem 2: browser crash `Cannot read properties of undefined (reading 'BYTES_PER_ELEMENT')` (Emscripten 5.0.7)

Symptom (Chrome console, right after the players log in to the signaling server):

```
Uncaught TypeError: Cannot read properties of undefined (reading 'BYTES_PER_ELEMENT')
    at _rtcCreatePeerConnection (atlas-party.js)
```

Cause: `deps/datachannel-wasm/wasm/js/webrtc.js:177` reads `Module['HEAPU32']` (and `Module['HEAPU8']` at lines 422 and 463). Since **Emscripten 4.0.7**, the memory views `HEAP8`, `HEAPU32`, etc. are no longer exported on `Module` by default, so the value is `undefined`.

Fix: pin Emscripten to **4.0.6**, the last release that still exports them. This also avoids Problem 1. (Alternative, not used: keep a newer version and add `-sEXPORTED_RUNTIME_METHODS=HEAPU8,HEAPU32` to the link flags.)

Problem 3: `IO_Error: not enough data` in `Shamir::get_hyper` (all 4 players)

Symptom (visible after building with `DEBUGGING=1`; before that it only showed `Uncaught 3964392`):

```
worker: onmessage() captured an uncaught exception: IO_Error: IO-Error : not enough data
    at octetStream::input(std::istream&)
    at Shamir<AtlasShare<gfp_<0,2>>>::get_hyper(...)
    at Shamir<...>::get_randoms(PRNG&, int)
    at AtlasPrep<...>::buffer_inputs(int)
```

Cause: ATLAS is built on Shamir secret sharing. `Shamir::get_hyper` (`Protocols/Shamir.hpp:202-229`) first tries to load a cached matrix file `Player-Data/Hyper-1-4-<prime>`. If the file is missing, it throws `IO_Error` inside a `try { ... } catch (...)` and then computes the matrix itself. That is normal behaviour. However, webSPDZ compiles without C++ exception support, and Emscripten's default is that exceptions can be thrown but never caught. So the expected throw aborted the program. (ATLAS seems not to have been tested by the webSPDZ authors: `atlas-party.x` is commented out in `Makefile:153`.)

Fix: enable native WebAssembly exceptions for all objects and the final link. `MY_CFLAGS` is included in `CFLAGS` (`CONFIG:140`), which is used for both compiling and linking:

```
echo 'MY_CFLAGS += -fwasm-exceptions' > CONFIG.mine
```

A full rebuild (`make -B`) is required so that every `.o` and `libwebSPDZ.so` uses the same exception mode.

4. Step-by-step: reproduce the working setup

4.1 Pin Emscripten to 4.0.6

```
cd ~/emsdk
./emsdk install 4.0.6
./emsdk activate 4.0.6
source ~/emsdk/emsdk_env.sh
emcc --version          # must print 4.0.6
```

Note: emsdk unpacks releases into the same `~/emsdk/upstream` folder, so installing 4.0.6 replaces the version that was there before. Run `source ~/emsdk/emsdk_env.sh` in **every** terminal you use.

4.2 Enable WebAssembly exceptions

```
cd ~/ProjectGuarnizo/SPDZ/webSPDZ
echo 'MY_CFLAGS += -fwasm-exceptions' > CONFIG.mine
```

4.3 Inputs and MPC program

```
printf '4\n13\n6\n10\n'  > Player-Data/Input-P0-0
printf '15\n3\n12\n7\n'  > Player-Data/Input-P1-0
printf '8\n17\n5\n11\n'  > Player-Data/Input-P2-0
printf '12\n8\n14\n4\n'  > Player-Data/Input-P3-0
./compile.py surplus_matching_private
ls Programs/Bytecode/surplus_matching_private-0.bc
```

Each file must have 4 values, one per line, ending with a newline.

4.4 Build ATLAS for the browser

```
source ~/emsdk/emsdk_env.sh
make -B atlas-party.x -j2
ls -lh atlas-party.html atlas-party.js atlas-party.wasm
```

The build packs `Player-Data` and `Programs` into the page. **Rebuild after any change to the inputs or the program.** Optional: add `DEBUGGING=1` to get readable exception messages in the console.

4.5 Start the servers (two terminals)

```
# Terminal A
python3 https-server.py localhost 8000

# Terminal B
source ~/emsdk/emsdk_env.sh
node signaling_wss_server.js localhost 8080
```

4.6 Open the 4 players in Chrome

```
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-
w,0,0,surplus_matching_private,-OF,.
```

```
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-
w,0,1,surplus_matching_private,-OF,.
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-
w,0,2,surplus_matching_private,-OF,.
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-
w,0,3,surplus_matching_private,-OF,.
```

After rebuilding, hard-reload each tab (**Ctrl+Shift+R**) so Chrome doesn't use a cached `.js` / `.wasm`.

5. Reading the console

- Lines like `Using statistical security parameter 40`, `[DataChannel open: ...]` and `All clients connected!` show in red only because Emscripten prints `std::cerr` with `console.error`. They are normal status messages.
- In the signaling terminal, `Disconnected ... All players have disconnected` after the offers and answers is normal: the signaling server is only needed to set up the WebRTC connections.
- `404 /.well-known/appspecific/com.chrome.devtools.json` on the HTTPS server comes from Chrome DevTools and can be ignored.
- Expected result per player: `YOUR PARTNER CODE = ...` (`-1` = no useful partner).

6. Version compatibility at a glance

Emscripten	**Link of `libwebSPDZ.so`**	**`Module['HEAPU32']` in webrtc.js**	**Result**
6.0.x (6.0.10 tested)	fails (-fPIC)	undefined	does not build
4.0.7 – 5.0.7 (5.0.7 tested)	OK	undefined	crashes in browser
4.0.6	OK	OK	**works (with `-fwasm-exceptions`)**